CSC2042S Assignment 3: Neural Networks on Fashion-MNIST-Student number: RHMABD006

1. Introduction

The goal of this assignment was to build a neural network that classifies images of clothing, and then to test how different hyperparameters change its performance. I used the Fashion-MNIST dataset, which has 70,000 grayscale images of size 28 by 28 pixels. There are 10 classes, such as T-shirt, Trouser, Coat and Sneaker I first built a baseline model and then tested 27 hyperparameter configurations, including the baseline settings, to see whether any configuration could improve its performance.

2. Design Decisions- Data. I downloaded the 60,000 image training set with torchvision. I split it randomly into 50,000 images for training and 10,000 for validation. I fixed the random seed at 42, so the split is the same every time. The 10,000 image test set was not used. **Normalisation.** Each image was converted to a tensor, which scales pixel values to between 0 and 1. I then normalised the pixels using the mean 0.2860 and the standard deviation 0.3530 of the Fashion-MNIST training data. Normalising keeps the inputs centred around zero, which helps gradient descent train faster and more steadily. **Architecture.** The model is a Multi-Layer Perceptron as required. It flattens each image into 784 numbers, then passes them through a hidden layer of 128 neurons with ReLU, a hidden layer of 64 neurons with ReLU, and an output layer of 10 neurons. The output layer gives raw scores, called logits, because the loss function applies softmax itself. I added an optional dropout setting to the class. When dropout is 0, the model is exactly the required baseline. **Training.** I used cross entropy loss, because this is a multi-class problem. I used the Adam optimiser, because it adapts the step size for each weight and works well without much tuning. The batch size was 64 and every model trained for 10 epochs. In each step the code clears old gradients, runs the forward pass, calculates the loss, runs backpropagation and updates the weights. After each epoch the code records the training loss, validation loss, training accuracy and validation accuracy. Validation and training accuracy are measured with dropout switched off. **Baseline.** The baseline uses a learning rate of 0.001 (the Adam default), no dropout and no weight decay.

Hyperparameters chosen. I chose three hyperparameters and gave each three values:

- Learning rate: 0.0005, 0.001 and 0.003. This is a learning process hyperparameter. It controls how big each weight update is.
- Dropout rate: 0.0, 0.2 and 0.4. This is a regularisation hyperparameter. It randomly switches off neurons during training so the network cannot depend too much on any single neuron.

- Weight decay: 0, 0.0001 and 0.001. This is also regularisation. It adds a penalty on large weights, which keeps the model simpler.

I tried every combination, which gives 3 x 3 x 3 = 27 runs. The grid includes the baseline settings, so the comparison is fair. Every run used the same seed, so all models started from the same initial weights.

3. Baseline Results-The baseline achieved a final validation accuracy of 89.29% and a final training accuracy of 92.81%. Its training loss decreased from 0.4968 in epoch 1 to 0.2157 in epoch 10. The validation loss reached its minimum of 0.3057 at epoch 8 and finished at 0.3109. Although the validation loss improved initially and then fluctuated around a similar level, the training loss continued to decrease. The gap between training and validation loss suggests that the model began to overfit as training progressed.

4. Experiment Results

Table 1: Final training accuracy (top 10 runs by validation accuracy)

Learning rate	Dropout	Weight decay	Training accuracy
0.001	0.0	0	92.81%
0.0005	0.0	0	92.47%
0.0005	0.0	0.0001	92.26%
0.001	0.0	0.0001	92.43%
0.0005	0.0	0.001	91.28%
0.0005	0.2	0	91.12%
0.0005	0.2	0.0001	90.83%
0.001	0.2	0	90.76%
0.001	0.0	0.001	90.23%
0.0005	0.4	0.0001	89.42%

Table 2: Final validation accuracy (top 10 runs)

Learning rate	Dropout	Weight decay	Validation accuracy	Change vs baseline
0.001	0.0	0	89.29%	+0.00
0.0005	0.0	0	89.23%	-0.06
0.0005	0.0	0.0001	88.98%	-0.31
0.001	0.0	0.0001	88.92%	-0.37
0.0005	0.0	0.001	88.87%	-0.42
0.0005	0.2	0	88.62%	-0.67
0.0005	0.2	0.0001	88.52%	-0.77
0.001	0.2	0	88.43%	-0.86
0.001	0.0	0.001	88.18%	-1.11
0.0005	0.4	0.0001	88.14%	-1.15

The worst-performing run used a learning rate of 0.003, dropout of 0.4 and weight decay of 0.001. It achieved 82.77% validation accuracy and 83.39% training accuracy.

5. Visualisations

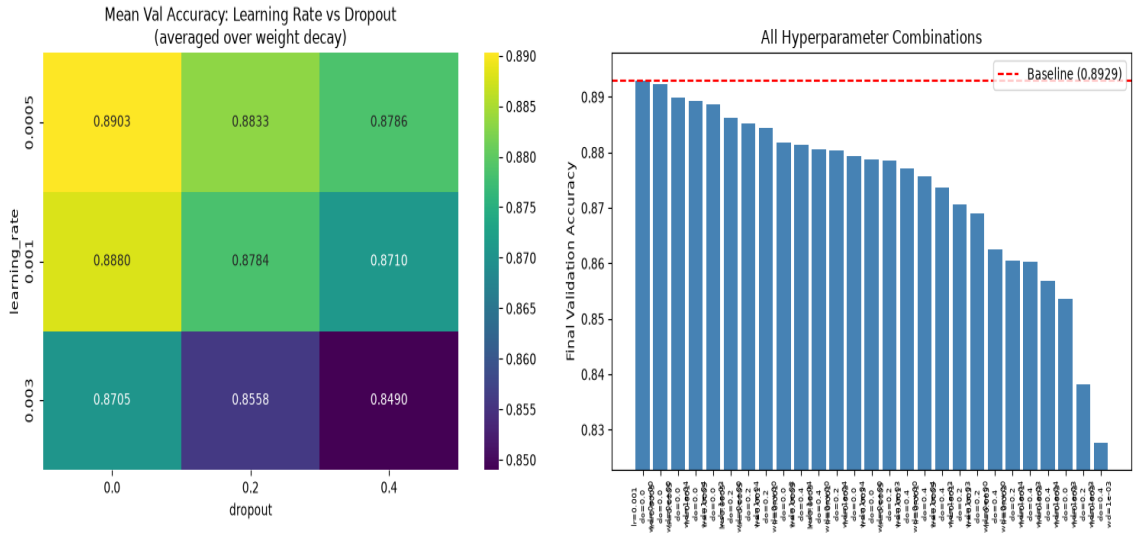


Figure 1: Left: mean validation accuracy for each learning rate and dropout pair, averaged over weight decay. Right: validation accuracy of all 27 runs, with the baseline shown as a red dashed line.

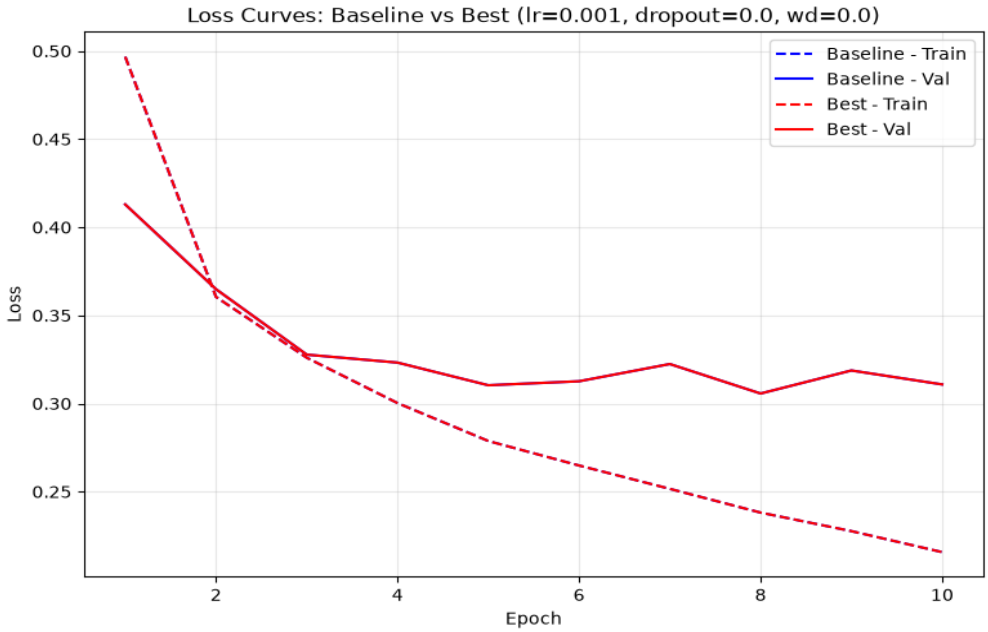


Figure 2: Training and validation loss curves for the best-performing configuration, which was the baseline (learning rate = 0.001, dropout = 0.0, weight decay = 0). The curves overlap with those of the baseline because the best model is the baseline configuration.

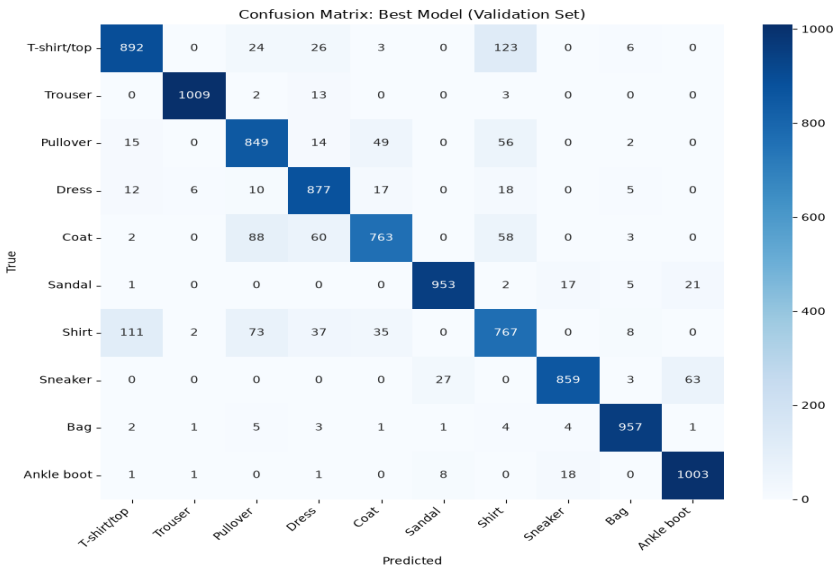


Figure 3: Confusion matrix of the best model on the validation set

6. Discussion-6.1 Best model versus baseline-The best-performing configuration was the baseline itself, with a learning rate of 0.001, dropout of 0.0 and weight decay of 0. It achieved 89.29% validation accuracy, meaning that hyperparameter tuning did not improve on the baseline. The runner-up configuration, with a learning rate of 0.0005, dropout of 0.0 and weight decay of 0, achieved 89.23%, just 0.06 percentage points lower. This small difference may reflect normal variation between runs rather than a meaningful improvement. Overall, none of the tested configurations clearly outperformed the baseline.

6.2 Which hyperparameter mattered most-Learning rate had the largest observed effect on mean validation accuracy. The mean accuracies were 88.41% for a learning rate of 0.0005, 87.91% for 0.001 and 85.84% for 0.003, a spread of 2.56 percentage points. The difference between the first two settings was relatively small, but the lower accuracy at 0.003 suggests that this learning rate was less suitable for generalisation. Dropout and weight decay had equal observed spreads of 1.67 percentage points. Mean accuracy decreased from 88.29% with no dropout to 87.25% with dropout of 0.2 and 86.62% with dropout of 0.4. Similarly, mean accuracy decreased from 88.04% with no weight decay to 87.76% with weight decay of 0.0001 and 86.37% with weight decay of 0.001. Too much dropout can make it harder for a small network to learn useful patterns, while excessive weight decay restricts the size of its weights and can prevent it from fitting the training data effectively within 10 epochs. This is consistent with the worst-performing configuration, which used a learning rate of 0.003, dropout of 0.4 and weight decay of 0.001, achieving only 82.77% validation accuracy and 83.39% training accuracy. However, a learning rate of 0.003 did not always prevent the model from fitting the training data: with no dropout or weight decay, one run reached 91.23% training accuracy. The results therefore suggest that combining a high learning rate with stronger regularisation was particularly harmful.

6.3 Overfitting analysis-The baseline shows some signs of overfitting. Its training loss decreased to 0.2157 by epoch 10, while its validation loss reached a minimum of 0.3057 at epoch 8 and finished at 0.3109. The final difference between validation and training loss was 0.0951. The baseline also achieved 92.81% training accuracy and 89.29% validation accuracy, giving an accuracy gap of 3.52 percentage points. Table 1 shows that some regularised configurations had smaller training-validation accuracy gaps, but this came at the cost of lower validation accuracy. For example, the configuration with a learning rate of 0.0005, dropout of 0.2 and weight decay of 0.0001 had an accuracy gap of 0.0231 and a validation accuracy of 88.52%, compared with the baseline's 89.29%. This suggests that regularisation reduced the gap for this configuration, but did not improve validation performance. One limitation is that training loss is measured with dropout enabled, whereas dropout is disabled during evaluation. Consequently, training and validation losses are not measured under exactly the same conditions, which should be considered when interpreting their difference.

6.4 Error analysis-The confusion matrix in Figure 3 shows that the model often confuses clothing items with similar appearances. T-shirt/top was misclassified as Shirt 123 times, while Shirt was misclassified as T-shirt/top 111 times. Coat was also misclassified as Pullover 88 times. These errors are understandable because the classes share similar shapes and features. In small, 28 × 28 grayscale images, details such as collars, buttons, zips and sleeve lengths can be difficult to distinguish. The model is a multilayer perceptron (MLP) that flattens each image into a vector, so it does not explicitly capture local spatial patterns in the way a convolutional neural network (CNN) does. This may make it harder to recognise subtle differences between clothing items. The confusion matrix suggests that distinguishing visually similar upper-body clothing is one of the model's main challenges. This assignment used a multilayer perceptron to classify Fashion-MNIST images and evaluated 27 hyperparameter configurations. The best configuration was the baseline, achieving 89.29% validation accuracy, so tuning produced no improvement over the baseline. Learning rate had the largest observed influence on mean validation accuracy, while stronger dropout and weight decay generally reduced performance. The baseline results showed a gap between training and validation performance, suggesting some overfitting, although further comparison with a different configuration is needed to determine whether tuning reduced it. Most classification errors occurred between visually similar clothing classes. Future work could explore a convolutional neural network to capture spatial features more effectively and repeat experiments with multiple random seeds to determine whether small performance differences are consistent.

AI declaration

I used Chat gpt and Claude and uploaded Agents.md to both. I used ai to help me understand certain parts of the assignment, fix syntax issued in my code and help keep track of experiments as well as recommendations, It helped explain concepts to me. I did the core part-Code and Report by myself.